Closed-Loop Threat-Guided Auto-Fixing of Kubernetes Container Security Misconfigurations

Brian Mendonca and Vijay K. Madisetti, *Fellow, IEEE*

Abstract—Containerized applications depend on Kubernetes YAML manifests that configure images, permissions, and storage; small configuration errors can introduce outages or security gaps. Most existing tools detect or block such problems but do not produce verified, ready-to-apply repairs. We built `k8s-auto-fix` to close the loop: detect an issue, suggest a small patch, verify it safely, and line it up for application. On a fixture-seeded 1,000-manifest replay against a live cluster, all verifier-accepted patches applied without rollback (1,000/1,000 dry-run/apply acceptance). On a 15,718-detection offline run, deterministic rules plus safety checks accepted 13,338 of 13,373 patched items (99.74%; auto-fix rate 0.8486; median patch length 9). An optional LLM mode reaches 88.52% acceptance on a 5,000-manifest corpus. In deterministic queue replay, a simple risk-aware scheduler cuts the worst-case wait for high-risk items by 7.9×. We release all data and scripts so others can reproduce these results.

Index Terms—Kubernetes, SAST, DAST, Admission control, Server-side dry-run, YAML, Pod Security, JSON Patch, Policy Enforcement, Kyverno, OPA Gatekeeper, Auto-fix, CI/CD, CVE, EPSS, Guidance retrieval, Risk-based scheduling

1 Introduction

Kubernetes has become the de facto operating system for the cloud, yet its declarative configuration model remains prone to security misconfigurations [32], [38]. A manifest is the text file that tells Kubernetes what to run, with which permissions, and on what resources. When these files are edited by hand, a stray `privileged: true`, a floating `:latest` image tag, or a missing `runAsNonRoot` line can quietly open a security gap or break a deployment.

Problem. Existing tooling addresses only part of this lifecycle. Static analyzers and admission controllers such as Kyverno and OPA Gatekeeper are effective at *detecting* or *blocking* insecure manifests, but they rarely produce a safe, ready-to-apply repair for an existing, non-compliant resource. Operators are therefore left to remediate by hand, which is slow and error-prone. Recent advances in Large Language Models (LLMs) suggest a path to automated repair and configuration validation [20], [34], but security studies show that LLM reasoning over vulnerabilities remains unreliable without external checks [31]. The open problem we target is *validated* remediation: turning a detected misconfiguration into a small patch that is proven safe before it is allowed near a cluster.

Approach. We present `k8s-auto-fix`, a closed-loop *detect* → *propose* → *verify* → *prioritize* pipeline whose proposer defaults to deterministic rules and admits optional LLM-generated patches only behind the same guardrails. The design is *threat-guided* in two concrete senses that recur through the pipeline. First, the verifier enforces a fixed set of security invariants—a policy re-check that the original violation is gone, schema validation, a server-side dry-run, and universal no-new-violation safety gates—so the threat a detection represents must be demonstrably removed before a patch is accepted. Second, the scheduler orders accepted work by a transparent risk score R built from policy severity, configured KEV-style boosts, and observed verifier priors; live CVE/EPSS feed joins are planned enrichment rather than required for the reported results. The acceptance contract is intentionally narrow: a patch is accepted only when it is policy-clean, schema-valid, API-admissible, and free of the universal safety regressions we check.

Contributions. This paper makes the following contributions:

- A closed-loop remediation pipeline for Kubernetes manifests that couples deterministic and optional LLM-based patch generation with a multi-gate verifier (policy re-check, schema validation, server-side dry-run, and universal safety invariants), so every accepted patch carries machine-checkable evidence that it removed the violation without introducing a new one.
- A threat-guided, risk-aware scheduler that prioritizes accepted patches by policy severity, configured KEV-style boosts, and observed verifier priors while an aging term bounds starvation, turning a flat remediation backlog into a risk-ordered queue.
- A reproducibility bundle for deterministic and LLM-backed modes on offline corpora and a fixture-seeded live cluster, with paper-facing summary tables regenerated by `make reproducible-report`

- *B. Mendonca is with the College of Computing, Georgia Institute of Technology, Atlanta, GA 30332 USA (e-mail: brian.mendonca6@gmail.com).*
- *V. K. Madisetti is with the School of Cybersecurity and Privacy, Georgia Institute of Technology, Atlanta, GA 30332 USA (e-mail: vkm@gatech.edu).*
- *Corresponding author: Vijay K. Madisetti.*

and per-claim command scope enumerated in ARTIFACTS.md.

Organization. Section 2 positions `k8s-auto-fix` against detection-only tooling, admission-time controllers, LLM-based repair, and emerging security agents. Section 3 describes the closed-loop design and traces two real manifests through every stage. Section 4 details the implementation, verification gates, and risk-bandit scheduler. Section 5.3 reports the evaluation, Section 6 states limitations and threats to validity, and Section 7 concludes with discussion and future work.

Legend (Table 1). A "bandit" scheduler balances risk, fairness, and expected time; "guardrails" are the safety checks (policy, schema, dry-run) plus simple sanitization before apply; "JSON Patch" is a small, surgical edit to the YAML; and "CRD/fixtures" are supporting Kubernetes objects the API expects (namespaces, service accounts, custom resources) that some admission tools require before they can mutate a file.

Metric caveat. Table 1 aggregates metrics reported by prior work that span admission latency, MTTR, and acceptance rates, so values are not strictly comparable; they provide qualitative context only.

Table 2 grounds those differences with shared per-policy slices, not a single aggregate leaderboard. Tables 1 and 2 therefore use different rosters on purpose: Table 1 compares representative systems by lifecycle role, while Table 2 is limited to tools with reusable policy-level artifacts or scanner interfaces for the same manifest slices. `k8s-auto-fix` lands 33–100% acceptance across targeted high-risk policies, while unsupported `no_host_ports` remains 0%. Some cells have small denominators, and Kyverno/Polaris rows use the larger detection sets their policies can process; we therefore treat the table as role/coverage evidence. Kyverno's mutate CLI reaches 100% on overlapping checks when fixtures are present, but admission can fail before mutation when they are absent. Polaris' CLI is detection-oriented, MAP depends on the Kubernetes CEL/JSONPatch admission-policy surface, and aligned LLMSecConfig prompts accept none of this slice. We do not read these results as "outperforming" Kyverno; our distinction is post-hoc repair of *existing* manifests plus per-patch verifier evidence.

2 Related Work

We group prior work into five categories—detection-only tooling, admission-time policy engines, LLM-based configuration repair, large-scale SRE automation, and emerging security agents—and state, for each, the gap that motivates a closed verification loop. Table 1 summarizes the comparison; the discussion below makes the categorization explicit.

1. Detection-only tooling. Static analyzers such as `kube-linter` [5] and dashboards such as Polaris [6] identify misconfigurations and score manifests against best-practice checks; GLITCH shows the broader value of technology-agnostic IaC security-smell detection [33]. Their output, however, is a finding rather than a validated, ready-to-apply patch. We reuse such detectors as the *Detector* stage and add the missing repair-and-verify stages on top.

2. Admission-time policy engines. Kubernetes admission control runs mutating logic before validating logic [3]; Kyverno [23] and OPA Gatekeeper [4] validate or mutate manifests at admission time, including Kyverno strategic-merge/JSONPatch and mutate-existing rules [24]. They are powerful guardrails, but they act primarily on the create/update path into the cluster and do not, by themselves, repair the large body of *existing*, non-compliant resources or rank outstanding work by risk; mutation also depends on the cluster already carrying the namespaces, service accounts, and CRDs an incoming manifest references. Our verifier reuses these engines for the policy re-check while operating post-hoc on stored manifests.

3. LLM-based configuration repair. GenKubeSec [21] uses LLM prompts to detect, localize, and explain Kubernetes misconfigurations and to suggest remediations; it reports strong detection precision/recall (we cite the authors' figures and did not reproduce their pipeline) but leaves the application and validation of fixes manual. Minna *et al.* mine Artifact Hub Helm charts, compare Checkov/KICS findings, and evaluate LLM-generated mitigations on the same ecosystem our corpus draws from [19]; Lian *et al.* study LLM-based configuration validation across systems [20]. LLMSecConfig [18] generates LLM repairs guided by scanner findings and policy identifiers, but its acceptance check is scanner-level and does not include a server-side dry-run or the universal safety invariants we enforce. These works make LLMs useful candidates, while the security-reasoning limits observed by Ullah *et al.* motivate treating model output as untrusted until external verification succeeds [31].

4. Large-scale SRE automation. Production fleet-management systems such as Borg [25] automate remediation and rollback at scale, but they target running infrastructure and operational toil rather than the static hardening of application manifests before deployment.

5. Emerging security agents. Concurrent with this work, OpenAI introduced Codex Security, formerly Aardvark, as a research-preview application-security agent [29], while KubeIntellect [30] and KubeLLM [22] propose LLM-orchestrated agents for Kubernetes management and troubleshooting. These agents target general software vulnerabilities or broad cluster operations; neither is centered on enforcing Kubernetes-specific configuration invariants (schema, policy re-check, and server-side dry-run) as the acceptance boundary for each patch. Codex Security is available as a research preview rather than as a public batch-evaluation interface for identical-manifest experiments, so a direct experimental comparison is not yet possible; we therefore position against it qualitatively and discuss a head-to-head agent comparison as future work in Section 7.

Positioning. Across these categories, prior tools cover individual stages—detect, block, or suggest—but rarely close the loop. The distinguishing acceptance boundary in `k8s-auto-fix` is that existing manifests, deterministic patches, and LLM-generated patches all pass through the same multi-gate verifier (policy re-check, schema validation, server-side dry-run, and universal safety invariants) before accepted work is ordered by auditable policy risk rather than first-in-first-out. This emphasis follows a broader APR

Table 1
Comparison of automated Kubernetes remediation systems (May 2026 source snapshot).

Capability	k8s-auto-fix (this work)	GenKubeSec [21]	Kyverno [23]	Borg/SRE [25]
Primary Goal	Closed-loop hardening (detect→patch→verify→prioritize)	LLM-based detection/remediation suggestions	Admission-time policy enforcement	Large-scale auto-remediation in production clusters
Fix Mode	JSON Patch (rules + optional LLM)	LLM-generated YAML edits	Policy mutation/generation	Custom controllers and playbooks
Guardrails	Policy re-check + schema + `kubectl apply -dry-run=server` + privileged/secret sanitization + CRD seeding	Manual review; no automated gates	Validation/mutation webhooks; assumes controllers	Health checks, automated rollback, throttling
Risk Prioritization	Bandit ($Rp/\mathbb{E}[t]$ + aging + KEV boost)	Not implemented	FIFO admission queue	Priority queues / toil budgets
Evaluation Corpus	15,718 detections (rules+guardrails: 13,338/13,373 patched = 99.74%; auto-fix 0.8486; median ops 9); 1,000 live-cluster manifests (100.0% dry-run/apply acceptance, seeded fixtures); 5,000 Grok manifests (88.52%); 1,264 supported manifests (100.00% rules, curated)	≈277k labeled KCFs; precision 0.990, recall 0.999 vs. RB tools; 30-sample expert validation of explanations/remediation (Malul et al.)	Admission-time policies and examples; no public comparable post-hoc repair corpus or acceptance percentage in the cited docs	Millions of production workloads (no public acceptance %)
Telemetry	Policy-level success probabilities, latency histograms, failure taxonomy	Token/cost estimates; no pipeline telemetry	Admission-controller reports and policy outcomes; no comparable proposer/verifier telemetry in the cited docs	MTTR, incident counts, operator feedback
Outstanding Gaps	Infrastructure-dependent rejects, operator study, scheduled guidance refresh in CI	Automated guardrails, risk-aware ordering	LLM-aware patching, risk-aware scheduling	Declarative manifest fixes, static analysis integration

Table 2
Per-policy baseline acceptance on shared security-context slices. Counts regenerate from data/detections.json, data/verified.json, and baseline CSVs under data/baselines/; denominators differ by tool/policy, so this is coverage evidence rather than one aggregate leaderboard.

Policy	k8s-auto-fix	Kyverno	Polaris (CLI)	Polaris (webhook)	MAP	LLMSecConfig
drop_cap_sys_admin	2/3 (66.7%)	n/a	n/a	n/a	1/3 (33.3%)	0/3 (0.0%)
drop_capabilities	1/2 (50.0%)	12/12 (100.0%)	0/12 (0.0%)	0/12 (0.0%)	1/2 (50.0%)	0/4 (0.0%)
env_var_secret	n/a	3/3 (100.0%)	0/3 (0.0%)	0/3 (0.0%)	n/a	n/a
no_host_path	1/1 (100.0%)	n/a	n/a	n/a	0/1 (0.0%)	0/1 (0.0%)
no_host_ports	0/1 (0.0%)	n/a	n/a	n/a	0/1 (0.0%)	0/1 (0.0%)
no_latest_tag	1/2 (50.0%)	25/25 (100.0%)	0/25 (0.0%)	0/25 (0.0%)	1/2 (50.0%)	0/2 (0.0%)
no_privileged	1/3 (33.3%)	5/5 (100.0%)	0/5 (0.0%)	0/5 (0.0%)	0/1 (0.0%)	0/1 (0.0%)
read_only_root_fs	2/3 (66.7%)	107/107 (100.0%)	0/107 (0.0%)	86/107 (80.4%)	1/3 (33.3%)	0/3 (0.0%)
run_as_non_root	2/3 (66.7%)	63/63 (100.0%)	0/63 (0.0%)	37/63 (58.7%)	1/3 (33.3%)	0/3 (0.0%)
set_requests_limits	4/6 (66.7%)	285/285 (100.0%)	0/285 (0.0%)	135/285 (47.4%)	1/3 (33.3%)	0/6 (0.0%)

lesson: scanner- or test-passing patches can still be overfit or unsafe unless the acceptance boundary checks the domain property the patch was meant to preserve [34]–[37]. All reported results are reproducible from the released data and scripts.

SAST vs. DAST positioning. Many tools only scan manifests offline (SAST). Our verifier additionally exercises a live API server with `kubectl apply -dry-run=server` [9] (DAST), which submits the request to the server without persisting it, so cluster-specific problems—missing CRDs, namespaces, or quotas—are caught before apply. This static-to-API acceptance path is what the live-cluster evaluation in Section 5.3 stresses.

3 System Design

We implement the closed loop *Detector* → *Proposer* → *Verifier* → *Scheduler* shown in Figure 1. Detectors produce structured JSON findings; the proposer applies rule-based

guards (with optional LLM backends) to emit minimal JSON Patches; the verifier enforces policy re-checks, schema validation, and `kubectl apply -dry-run=server`; and the scheduler orders work using risk-aware bandit scoring. Each stage persists artifacts (detections, patches, verified outcomes, queue scores), enabling reproducible evaluation (Section 5.3).

3.1 Notation

We model a manifest as a JSON/YAML document m and a detection as a tuple $d = (m, \pi, v)$, where π is the violated policy identifier and v is the violation evidence emitted by the detector. The proposer produces a JSON Patch δ (RFC 6902 [8]), and applying it yields the patched manifest $m' = \delta(m)$. A detection is *resolved* when m' satisfies the acceptance predicate $\mathrm{Acc}(m', \pi) = \mathrm{policy}(m', \pi) \wedge \mathrm{schema}(m') \wedge \mathrm{dryrun}(m') \wedge \mathrm{safe}(m')$, i.e. the conjunction of the four verifier gates defined in Section 4. The scheduler then ranks resolved items using a risk score R_i for queue item i, an empirical verifier success probability p_i for its policy, the observed proposer+verifier latency $\mathbb{E}[t_i]$, the accumulated queue age wait_i, and a configured KEV-style boost kev_i when the policy is flagged in the current artifact. Unless otherwise noted, wait times are reported in hours and fairness statistics (Gini, starvation rate) are computed over these waits. Section 4 gives the closed forms of R_i and the scheduling score S_i.

Detector disagreement and false positives. The detector runs `kube-linter` and a policy engine (Kyverno/OPA) and takes the *union* of their findings; a patch must satisfy *both* engines during verification, so the union is conservative for recall while the verifier remains the precision boundary. False positives are handled structurally rather than heuristically: every proposer fix is guarded by JSON Pointer existence checks, so a finding that does not correspond to a real field produces no edit (an empty patch) rather than a spurious change, and the policy re-check gate $\mathrm{policy}(m', \pi)$ rejects any patch that does not actually clear the asserted violation. A detection the proposer cannot resolve under these guards is never silently "fixed"; it is queued for human review.

Attempts and retry budget. An *attempt* is one proposer→verifier cycle for a single detection: the proposer emits a candidate patch and the verifier evaluates $\mathrm{Acc}(m', \pi)$. If an attempt fails, the failing gate and error trace are fed back and the proposer may retry, up to a configurable budget `max_attempts` (default 3, set in `configs/run.yaml`). After the budget is exhausted the item is dropped from automated handling and routed to review. Per-attempt latency and outcome are logged to `data/policy_metrics.json`, which supplies the online estimates p_i and $\mathbb{E}[t_i]$ the scheduler consumes alongside KEV flags.

3.2 End-to-End Walkthrough on Real Manifests

To make the closed-loop pipeline concrete, we trace two real-world manifests from the repository's test suite through each stage, from detection to scheduling. The goal is to demonstrate safe, automated remediation with full reproducibility and verifiable risk reduction.

Case 1: Remediating a Privileged Pod with a `:latest` Image Tag

This example, drawn from data/manifests/001.yaml, shows a common but high-risk pattern: a privileged container using a floating tag.

```
apiVersion: v1
kind: Pod
spec:
  containers:
  - name: app
    image: acme/api:latest
    securityContext:
      privileged: true
      allowPrivilegeEscalation: true
      capabilities: { add: ["SYS_ADMIN", "NET_ADMIN"] }
```

Walkthrough. We scan the file, draft a small fix, run three verification gates (policy, schema, server-side dry-run), and only then queue it for apply. "Accepted" here means policy- and API-admissible, not a guarantee of runtime workload correctness. The patch is idempotent—applying it twice does not duplicate fields or corrupt the file.

1. Detect (Union): The detector consumes this manifest and reports four policy violations: `no_privileged`, `drop_capabilities`, `run_as_non_root`, and `no_latest_tag`. These are emitted as the detector's structured detection report.

2. Propose (Rules Engine): The proposer consumes the detection report and generates a minimal JSON Patch that is idempotent—running it twice will not add duplicate fields. The pinned tag shown here (`acme/api:1.42.0`) is illustrative: the deterministic fixer requires an operator-configured replacement tag or image digest for the `no_latest_tag` policy and otherwise queues the item for human review rather than inventing a version. The resulting patch is as follows:

```
[
 {"op":"replace","path":"/spec/containers/0/
    securityContext/privileged","value":false},
 {"op":"replace","path":"/spec/containers/0/
    securityContext/allowPrivilegeEscalation","value":
    false},
 {"op":"remove","path":"/spec/containers/0/securityContext
    /capabilities/add"},
 {"op":"add","path":"/spec/containers/0/securityContext/
    capabilities/drop","value":["ALL"]},
 {"op":"add","path":"/spec/containers/0/securityContext/
    runAsNonRoot","value":true},
 {"op":"replace","path":"/spec/containers/0/image","value
    ":"acme/api:1.42.0"}
]
```

3. Verify (Triad): The verifier applies this patch to a in-memory copy of the manifest and runs it through the full triad:

- **Policy Re-check**: Passes, as the patched manifest no longer violates the four detected policies.
- **Schema Validation**: Passes, confirming the patch produces a structurally valid Kubernetes object.
- **Server Dry-Run**: Succeeds, as `kubectl apply -dry-run=server` reports the manifest would be accepted by the API server in a Kind cluster seeded with necessary fixtures.

The successful outcome is recorded in the verifier's structured log.

4. Schedule (Risk-Bandit): The scheduler gives the verified patch a high score because the risk is high (a privileged container), the fix is likely to succeed (based on prior outcomes),

and it is quick to apply. Higher scores move earlier in the queue.

Case 2: Hardening a Worker Pod with a `hostPath` Mount

```
spec:
  containers:
  - name: worker
    securityContext: { readOnlyRootFilesystem: false }
    resources: {}
  volumes:
  - name: host
    hostPath: { path: "/var/run/docker.sock" }
```

This second case, from data/manifests/002.yaml, targets three additional misconfigurations: a writable root filesystem, a dangerous `hostPath` volume mount, and missing resource requests and limits.

1. Detect: The detector flags `read_only_root_fs`, `no_host_path`, and `set_requests_limits`.

2. Propose: The rules engine generates a patch to harden the filesystem, replace the disallowed `hostPath` source with an `emptyDir`, and enforce resource quotas:

```
[
 {"op":"replace","path":"/spec/containers/0/
     securityContext/readOnlyRootFilesystem","value":true},

 {"op":"remove","path":"/spec/volumes/0/hostPath"},
 {"op":"add","path":"/spec/volumes/0/emptyDir","value
     ":{}},
 {"op":"add","path":"/spec/containers/0/resources","value
     ":{"requests":{"cpu":"100m","memory":"128Mi"},"limits
     ":{"cpu":"500m","memory":"256Mi"}}}
]
```

3. Verify: The verifier confirms the patch is valid for this smoke fixture: the triggering `no_host_path` assertion is gone, the YAML remains structurally valid, and the server-side dry-run accepts the object. The safety guardrails are critical here: had the `hostPath` mount been on an allowlisted path (e.g., for a metrics agent), the verifier would have preserved it. The same policy re-check discipline is what catches the ablation escapes in Section 5.3; the verifier does not treat a plausible, schema-valid patch as sufficient unless the original invariant is actually cleared.

4. Schedule: This item receives a moderate risk score. While `hostPath` is a serious issue, it is less critical than a privileged container. The patch is scheduled after higher-priority items, demonstrating the risk-aware nature of the queue.

Comparison with existing approaches. Kyverno (mutation) focuses on admission-time defaults and does not enforce a multi-gate verifier (policy+schema+server dry-run) prior to apply, depending on cluster fixtures for success and requiring bespoke policies plus controller context for complex hardening such as dropping `ALL` capabilities or removing privilege. GenKubeSec localizes and explains issues but leaves remediation and validation manual, offering no guaranteed JSON Patch and no dry-run alignment. LLMSecConfig generates LLM repairs with scanner checks but lacks our triad's server-side dry-run and hard safety invariants, which are key to preventing regressions in production-like clusters.

Safety and performance. The policy re-check gate prevented four escapes in ablation (Table 16); live-cluster replay achieved 100% dry-run/apply acceptance with zero rollbacks on the fixture-seeded subset. The scheduler prioritizes risk (P95 wait from 102.3 h to 13.0 h), closing the highest-impact items first under bounded budgets.

Table 3 makes the lifecycle gap explicit: comparable systems can detect, mutate, or suggest repairs, but they differ sharply on the evidence an operator receives before trusting a patch. We use "safety invariants" below for universal no-new-violation gates beyond confirming that one scanner finding disappeared.

3.3 Research Questions and Findings

We organize the evaluation around four research questions. The design makes a specific bet—that verification, not the generator, should be the acceptance boundary—and each question probes one consequence of that bet: whether the loop actually accepts useful fixes (RQ1), whether risk-aware ordering buys anything over the naive queue an operator would otherwise use (RQ2), whether that ordering starves low-risk work (RQ3), and whether the accepted patches are small enough to review and trust (RQ4).

RQ1 (Robustness): does the closed loop accept fixes across corpora and modes? *Rationale:* a remediation loop is only useful if a large fraction of detections survive all four gates, and that fraction should not collapse when the proposer switches from deterministic rules to an LLM backend. *Finding:* the loop delivers 88.52% acceptance on the Grok-5k sweep, 100.00% on the supported 1,264-manifest corpus in rules mode, and 13,338/13,373 (99.74%) accepted on the full 15,718-detection run under deterministic rules plus guardrails (auto-fix rate 0.8486 over detections; median 9 operations), with no `hostPath`-related safety failures remaining.

RQ2 (Scheduling effectiveness): does risk-aware ordering reduce the wait for dangerous items? *Rationale:* operators triage a backlog, so the metric that matters is not throughput—which is bounded by proposer/verifier latency regardless of order—but how long the highest-risk items wait. *Finding:* the bandit ($Rp/\mathbb{E}[t]$ + aging + KEV boost) reduces top-risk P95 wait from 102.3 hours under FIFO to 13.0 hours (7.9×) at comparable throughput.

RQ3 (Fairness): does prioritization starve low-risk work? *Rationale:* any priority queue risks indefinitely deferring low-risk items, so the aging term must be shown to bound starvation rather than merely shift it. *Finding:* aging keeps the median rank of the top-50 high-risk items at 25.5 while still draining lower-risk items; starvation is zero in the bounded parameter sweeps and falls from 93.4% to 19.1% for high-risk items in the longer operator replay.

RQ4 (Patch quality): are the accepted patches small and stable? *Rationale:* a patch that an operator must hand-audit is only reviewable if it is minimal and does not drift under reapplication. *Finding:* generated JSON Patches remain small (median 9 operations on the full 15,718-detection corpus; 5–8 on the smaller curated slices) and idempotent under repeated application.

4 Implementation and Metrics

Our system is designed as a linear pipeline with strict verification gates to ensure the safety and correctness of all proposed patches.

Table 3
Lifecycle contract for remediation tools: what evidence exists before an operator can trust a patch.

Lifecycle question	k8s-auto-fix (this work)	Kyverno	GenKubeSec	LLMSecConfig
When does it act?	Post-hoc on stored manifests and replayed live resources	Admission-time create/update path	Offline localization and explanation	Offline LLM repair attempt
What patch artifact exists?	Minimal RFC 6902 JSON Patch plus verifier evidence	Mutation policy result when a policy is installed	Textual remediation guidance	YAML edit proposed by the model
What accepts the patch?	Policy re-check + schema validation + `kubectl` server dry-run + safety invariants	Admission controller outcome in the target cluster	Human/manual validation	Scanner re-checks; no server dry-run or hard safety invariants
How is work ordered?	Risk-aware score $Rp/\mathbb{E}[t]$ with aging and KEV-style boosts	Admission arrival order	None	None

Reproducing the results. The full pipeline is driven by three stages—detection, patch proposal, and verification—invoked as `make detect`, `make propose`, and `make verify` from the repository root. The smoke corpus completes in a few minutes; full-corpus regeneration (over 15,000 detections) takes roughly 20–30 minutes on a laptop (Table 5 reports the environment). **Scalability considerations.** The end-to-end pipeline sustains millisecond-scale proposer latency and sub-second verifier latency on the supported corpus (Table 12); the scheduler replays thousands of queue items using persisted telemetry (see data/scheduler/) without recomputing detections.

4.1 The Closed-Loop Pipeline

The workflow consists of four stages: the **Detector** ingests a Kubernetes manifest and uses both `kube-linter` and a policy engine (Kyverno/OPA) to identify violations, taking the union of all findings; the **Proposer** takes the manifest and violation data and generates a JSON Patch, defaulting to deterministic rules for the covered policy families (image tags, privilege, capabilities, non-root execution, read-only root filesystems, host namespace/mount controls, seccomp, and resource requests/limits) while guarding each operation with JSON Pointer existence checks and enforcing minimality/idempotence via `tests/test_patch_minimality.py`; the **Verifier** applies the patch to a copy of the manifest and subjects it to the verification gates described below, recording evidence in `data/verified.json`; and the **Budget-aware Retry** loop uses a configurable retry budget (`max_attempts` in `configs/run.yaml`, default 3) so the proposer can reattempt if verification fails, logging the error trace for inspection.

4.2 Verification Gates

To be accepted, a patched manifest must pass a multi-layered verification process. The **Policy Re-check** re-evaluates the patched manifest with the same policy logic that triggered the violation, using explicit assertions for each covered policy (e.g., `no_latest_tag`, `no_privileged`, `drop_capabilities`, `drop_cap_sys_admin`, `run_as_non_root`, `read_only_root_fs`, `no_host_*` flags, `no_allow_privilege_escalation`, `enforce_seccomp`, `set_requests_limits`); the detector hook for re-scanning is available via `-enable-rescan`, and non-allowlisted `hostPath` volumes are auto-remediated to `emptyDir: {}` in guardrails to satisfy the universal safety gate. The **Schema Validation** step applies the JSON Patch via `jsonpatch` and rejects malformed paths or operations, surfacing issues to the retry loop. The **Server-side Dry-run** uses `kubectl apply -dry-run=server`, when available, to simulate how the Kubernetes API server would handle the change, marking failures as not accepted and persisting CLI output for analysis. Finally, the **No-New-Violations Safety Gates** enforce universal security assertions to prevent regressions: they block `privileged: true` in any container; reject `capabilities.add` entries containing `NET_RAW`, `NET_ADMIN`, `SYS_ADMIN`, `SYS_MODULE`, `SYS_PTRACE`, or `SYS_CHROOT`; flag empty `capabilities.drop` lists when capabilities are present; require each container to specify a non-empty image; and restrict host mounts to approved paths (/var/run/secrets/kubernetes.io/serviceaccount, /var/lib/kubelet/pods, /etc/ssl/certs), rewriting non-allowlisted `hostPath` volumes to `emptyDir: {}` by guardrails.

Fairness in Action. To illustrate how the scheduler's aging mechanism prevents starvation, consider three items in a simplified queue: Item A is a high-risk privileged container with a configured KEV-style boost, Item B is a medium-risk container with a ':latest' image tag, and Item C is a low-risk container with missing resource limits. Initially, Item A has the highest score and is processed first, but as Items B and C wait their increasing 'wait' time boosts their scores. This "aging" ensures that even low-risk items are eventually processed instead of being indefinitely starved by a constant stream of high-risk items, a fairness target shared with constrained bandit formulations [28]. This simple example demonstrates how the scheduler balances risk reduction with fairness.

5 Implementation Status and Evidence

Table 4 ties each pipeline stage to the concrete code and artifacts currently in the `k8s-auto-fix` repository. The implementation operates end-to-end in rules mode without external API dependencies; LLM-backed modes are configurable and evaluated off-line, while the default reproducible path uses rules mode. **Operational integration.** The pipeline maps directly onto a CI/CD workflow: the detection, proposal, and verification stages run as sequential steps, fixtures are published once per environment, and operator feedback is collected from the verifier output. A containerized configuration reproduces the same stages in a

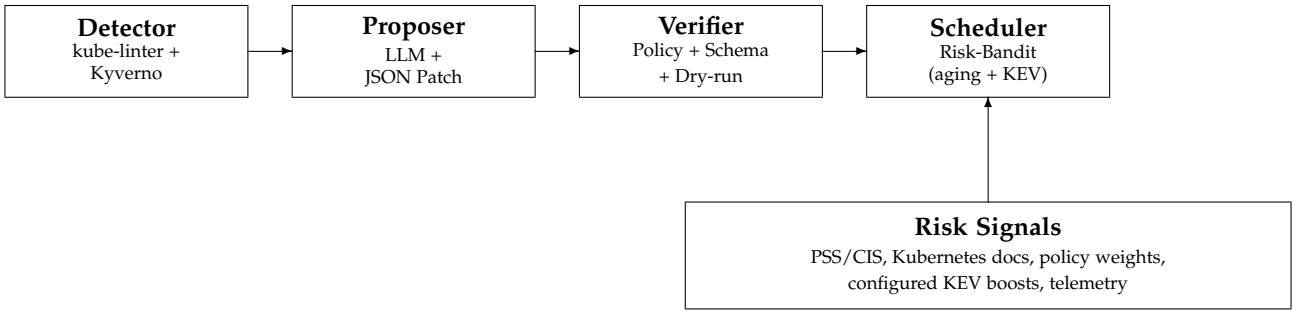


Figure 1. Closed-loop architecture with detector, proposer, and verifier gates (policy re-check, schema validation, `kubectl apply -dry-run=server`) feeding the risk-aware scheduler. The scheduler consumes `policy_metrics.json` entries $\{p, \mathbb{E}[t], R, \text{KEV}\}$ to score work using the scheduling function. Evaluated component versions: `kube-linter` 0.7.6, Kyverno CLI (Kind staging), `kubectl` 1.34.1, `kind` 0.30.0, Kubernetes 1.34.0, and the Grok/xAI `grok-4.3` proposer for LLM-backed runs (full toolchain in Table 4).

self-contained image for hermetic evaluation. Full reproduction instructions and the adoption checklist are provided with the released artifact.

5.1 Detection Records and Test Evidence

Each detector record carries the fields {`id`, `manifest_path`, `manifest_yaml`, `policy_id`, `violation_text`}; the embedded `manifest_yaml` decouples downstream stages from the filesystem, so the proposer and verifier operate on the record alone. A representative record and its full schema ship with the released artifact (`data/detections.json`).

The test suite (`make test`) covers detector contracts, proposer guards, verifier gates, scheduler ordering, and patch idempotence, with generated patch-minimality checks gated behind `make artifact-test`. Beyond the deterministic contract tests, `tests/test_property_guards.py` runs property-based checks over hundreds of randomized manifests per run, confirming that the per-policy patchers add the expected hardening (dropping dangerous capabilities, denying privilege escalation, enforcing `RuntimeDefault` seccomp, preferring non-privileged defaults) and remain idempotent without widening the universal verifier gate beyond the invariants of Section 4.

5.2 Dataset and Configuration

Two deliberately vulnerable manifests (`001.yaml`, `002.yaml`) are retained for smoke tests, but all evaluation numbers in this report come from the much larger Grok corpus (5,000 manifests mined from ArtifactHub [26]) and the "supported" corpus (1,264 manifests curated after policy normalization). `configs/run.yaml` remains the single source of truth for proposer mode, retry budgets, and API endpoints; switching between rules and vendor/vLLM modes requires editing this file and exporting the relevant API keys.

Table 5 summarizes the runtime environment used for the regenerations in Section 5.3; the full dependency snapshot (including transient packages) resides in `data/repro/environment.json`. The supplemental appendix documents the ArtifactHub mining pipeline and the manifest hash corpus that underpins the datasets. Table 10 shows how fixture seeding collapses the verifier failure categories across the supported corpus.

Table 6 lists the Grok/xAI proposer settings that drive the API-backed sweeps.

5.3 Evaluation Results

All results in this section derive from the deterministically reproducible `rules` pipeline unless explicitly noted. Table 12 consolidates acceptance and latency statistics for each corpus. The API-backed Grok mode is likewise benchmarked (4,426 / 5,000 accepted; see data/batch_runs/grok_5k/metrics_grok5k.json) but requires external credentials and funded access, so we treat it as an opt-in configuration rather than the default reproduction path. Consolidated metrics (acceptance + latency) live in data/eval/unified_eval_summary.json. Table 8 states, for each headline rate, the exact unit and corpus it is computed over, so acceptance, auto-fix, and risk figures are not conflated.

Detector accuracy. Running `scripts/eval_detector.py` on a synthetic nine-policy hold-out set confirms basic detector functionality with perfect precision and recall (Table 9). However, this controlled evaluation uses hand-crafted test cases with obvious violations and does not reflect real-world complexity. The live replay validates remediation *safety* for the items the detector flags—every accepted patch applied cleanly with zero rollback (data/live_cluster/results_1k.json; summary in data/live_cluster/summary_1k.csv)—but it does not measure detector *recall*. Detector recall on uncurated, real-world manifests beyond the labeled slices remains an open limitation.

ArtifactHub slice. To test against less curated input, we heuristically labelled 69 ArtifactHub manifests covering four common policies (`no_latest_tag`, `no_privileged`, `no_host_path`, `no_host_ports`). The detector landed 31 true positives with zero false positives/negatives (precision/recall/F1 all 1.0). Scoring is restricted to these policies (detections filtered via data/eval/artifacthub_sample_detections_filtered.json).

1. All paths are relative to the project root.
2. Artifacts live under data/ after running the corresponding `make` targets.

Table 4
Evidence for each stage of the implemented pipeline (current snapshot).

Stage	Implementation[1]	Artifacts Produced[2]
Detector	src/detector/detector.py src/detector/cli.py	Records in data/detections.json with fields {id, manifest_path, manifest_yaml, policy_id, violation_text}; seeded by data/manifests/001.yaml and 002.yaml.
Proposer	src/proposer/cli.py src/proposer/model_client.py, src/proposer/guards.py	data/patches.json containing guarded JSON Patch arrays. Rules mode emits single-operation fixes; vendor/vLLM modes require OpenAI-compatible endpoints configured in configs/run.yaml.
Verifier	src/verifier/verifier.py src/verifier/cli.py	data/verified.json logging accepted, ok_schema, ok_policy, and patched_yaml. Current policy checks assert the triggering policy (e.g., no_latest_tag, no_privileged, drop_capabilities, drop_cap_sys_admin, run_as_non_root, read_only_root_fs, no_host_* flags, no_allow_privilege_escalation, enforce_seccomp, set_requests_limits).
Scheduler	src/scheduler/schedule.py src/scheduler/cli.py	data/schedule.json with per-item scores and components {score, R, p, Et, wait, kev}; risk constants presently keyed to policy IDs.
Automation	Makefile	Reproducible commands for each stage: make detect, make propose, make verify, make schedule, make e2e.
Testing	tests/	make test invokes python -m pytest -q over the test suite, covering detector contracts, proposer guards, verifier gates, scheduler ordering, artifact helpers, docs-link drift checks, and patch idempotence. Optional generated-artifact checks run separately with make artifact-test.
Runtime Toolchain Versions (Evaluation Environment)		
Environment	Python 3.12.4 kubectl 1.34.1 kube-linter 0.7.6 kind 0.30.0	Kubernetes cluster: 1.34.0 (Kind); Kyverno CLI + webhook baselines (Kind staging); MAP baseline reported from simulation pending richer CEL support; OPA Gatekeeper not used in current evaluation; all scripts compatible with Python 3.10+

Table 5
Execution environment for the reproduced rule-mode evaluations.

Component	Version
Python	3.12.4 (macOS-26.0-arm64)
jsonpatch	1.33
numpy	1.26.4
pandas	2.2.3

Table 6
LLM-backed proposer configuration for Grok/xAI sweeps (values from configs/run.yaml).

Parameter	Value
Model	grok-4.3
Endpoint	https://api.x.ai/v1/chat/completions
Temperature	0.0 (low variance)
Top-p	Provider default (unchanged)
Max tokens	Provider default ($<$ 1k-token patches)
Retries per call	2 (max attempts $=$ 3)
Timeout	60 s per request
Seed	1337 (workload seed; API may drift)

Labels, detections, and metrics live under data/eval/artifacthub_sample_labels.json, data/eval/artifacthub_sample_detections.json, and data/eval/artifacthub_sample_metrics.json.

The evaluation campaigns span deterministic and LLM-backed modes. Rules mode repairs 1,264/1,264 supported manifests (100%) with 29 ms median proposer latency and 242 ms median verifier latency (P95 517.8 ms), and scales to 4,677/5,000 accepted patches (93.54%) on the extended 5k corpus. The Grok/xAI proposer delivers 4,426/5,000 remediations (88.52%) with median JSON Patch length 9; telemetry records 4.38M input, 0.69M visible completion, and 11.40M total tokens including xAI-reported reasoning tokens, letting readers recompute dollar cost from current pricing [10]. On the latest full rules+guardrails run (15,718 detections), the pipeline accepts 13,338/13,373 patched items (99.74%; auto-fix rate 0.8486 over detections) with median patch length 9. Table 6 lists the Grok/xAI configuration, and Figure 4 contrasts deterministic throughput with Grok/xAI's API-driven variance.

To ground deployability we instrumented Grok/xAI proposer and verifier traces from the original 200-manifest

Table 7
Top 10 Grok/xAI dry-run failure causes

Failure Cause	Count
kubectl dry-run failed: error: error when retrieving current configuration of: Resource: "batch/v1, . . .	65
kubectl dry-run failed: error: error when retrieving current configuration of: Resource: "/v1, Resou. . .	20
kubectl dry-run failed: The ReplicationController "zulip-1" is invalid: * spec.template.spec.contai. . .	18
kubectl dry-run failed: Error from server (BadRequest): error when creating "STDIN": Deployment in v. . .	16
can't remove a non-existent object 'clusterName'	16
kubectl dry-run failed: Error from server (BadRequest): error when creating "STDIN": ReplicaSet in v. . .	14
kubectl dry-run failed: The StatefulSet "mysql" is invalid: * spec.template.spec.containers[0].volu. . .	14
kubectl dry-run failed: The Deployment "testground-daemon" is invalid: spec.template.spec.containers. . .	12
kubectl dry-run failed: The CronJob "tf-r2.4.0-keras-api-custom-layers-v2-32" is invalid: spec.jobTe. . .	11
kubectl dry-run failed: The CronJob "tf-nightly-retinanet-func-v3-8" is invalid: spec.jobTemplate.sp. . .	11

replay (data/batch_runs/grok200_latency_summary.csv, data/batch_runs/verified_grok200_latency_summary.csv). The proposer shows 5.10 s median end-to-end latency (P95 16.9 s), while the verifier adds 89.5 ms median latency (P95 369.3 ms). Failure causes remain dominated by dry-run contract mismatches and legacy StatefulSets; Table 7 summarises the top categories and ties each mitigation to a regression class. These latency medians are scoped to the archived 200-manifest trace, while full 5k acceptance and token telemetry are reported separately.

The failure taxonomy (Table 7, sourced from data/grok_failure_analysis.csv) shows that the largest Grok dry-run failure category (65 cases) stems from the Kubernetes API refusing to return the existing object (common for CRDs that require elevated RBAC), 20 cases arise from core/v1 resource lookups with stale UIDs, and the remaining long tail is dominated by invalid StatefulSet/CronJob specs. These concrete counts shaped the mitigations we now ship: the live replay seeds every CRD+RBAC pair in data/live_cluster/crds/, StatefulSets go through a schema pre-flight that patches missing `volumeMounts`, and we block retries on dry-run errors that originate from immutable fields (instead queueing the manifest for human review). All of these safeguards are enforced uniformly for both the rules and Grok pipelines.

Figure 3 provides additional context: Kyverno's admission-time hooks excel when fixture seeding succeeds, but our post-hoc verifier keeps acceptance steady even when controllers are absent. Figure 5 then shows how the

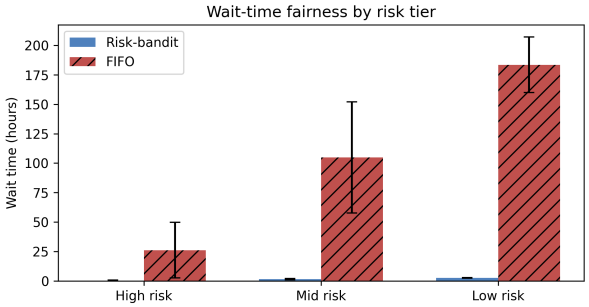


Figure 2. Median wait time (bars) and P95 error bars for each risk tier. Bandit scheduling keeps the top quartile under 0.7 h while FIFO defers the same items for 26–50 h, illustrating the fairness gains summarized in data/scheduler/metrics_schedule_sweep.json and data/scheduler/metrics_sweep_live.json.

simulated scheduler variants balance acceptance and wait time, while the larger queue replay demonstrates the 7.9× reduction in top-risk P95 wait. Every figure in the evaluation section carries an accompanying explanation rather than standing alone.

Live-cluster replay on a stratified 1,000-manifest subset in the fixture-seeded Kind/Kubernetes 1.34.0 evaluation environment achieves 100.0% dry-run/apply acceptance (1,000/1,000) with full alignment between server-side dry-run and apply on this subset. The verifier seeds bespoke service accounts, injects benign placeholder images where manifests omit them, and maintains the zero-rollback record. Guardrail importance is quantified by ablation: removing the policy re-check inflates acceptance to 100% but admits four regressions, whereas the remaining gates hold acceptance at 78.9% with zero escapes (Table 16).

Risk-aware scheduling reduces queue latency for high-risk items. Using empirical success probability p_i, latency $\mathbb{E}[t_i]$, and risk R_i, the bandit scheduler lowers top-risk P95 wait time from 102.3 h (FIFO) to 13.0 h while keeping the median rank of the top 50 items at 25.5. The risk-tier replay in Figure 2 shows the same effect visually: high-risk work waits less than an hour with bandit ordering yet idles for 26–50 h under FIFO. The longer operator replay in data/scheduler/fairness_metrics.json preserves the directional result: high-risk median wait is 14.75 h for bandit versus 130.33 h for FIFO, and starvation drops from 93.4% to 19.1%. Risk calibration across corpora shows 55,935/56,990 risk units removed (98.15%) on the supported dataset and 227,330/242,300 units (93.82%) on the 5k sweep, sustaining throughput near 4.5–4.9 risk units per expected-time interval (Table 11). Operator A/B replays yield 1,259 assignments per arm and confirm comparable acceptance and mean risk closed between bandit and FIFO (42.97 vs. 43.40, data/operator_ab/summary_staging.csv), with the fairness gain coming from which items are served first.

The queue replay in data/scheduler/fairness_metrics.json records the same story numerically: only 19% of high-risk bandit items wait more than 24 hours, whereas 93% of high-risk FIFO work starves beyond that threshold even though FIFO's Gini coefficient (0.28) appears superficially lower than our bandit run (0.34). We therefore report both Gini and starvation to show that categorical

Table 8
Metric denominators. Each headline number is computed over a specific unit and corpus; we state these explicitly to avoid ambiguity across acceptance, auto-fix, and risk metrics.

Dataset	Unit	Count	Mode	What the metric measures
Live replay	manifests	1,000	rules + seeded fixtures	dry-run/apply acceptance with zero rollback
Supported corpus	manifests	1,264	rules	per-manifest acceptance (curated after policy normalization)
Supported risk set	detections	1,278	rules + risk scheduler	policy-weighted risk removal (ΔR)
Full corpus (detections)	detections	15,718	rules + guardrails	total detections found
patched subset	patches	13,373	rules + guardrails	patches attempted
accepted subset	patches	13,338	rules + guardrails	verifier-accepted (99.74% of attempted)
Grok/xAI slice	manifests	1,313	LLM (opt-in)	paired rules-vs-Grok slice acceptance (100.00%)
Grok-5k	manifests	5,000	LLM (opt-in)	LLM patch acceptance (88.52%)

Definitions. "Acceptance" is computed over *attempted* patches; "auto-fix rate" (0.8486) is computed over *all* detections; live-replay "success" means server-side dry-run/apply alignment with zero rollback; "risk removal" uses policy-weighted detections. The curated corpora are normalized to supported policies, so per-manifest acceptance on those sets is not directly comparable to detection-level rates on the full corpus.

Table 9
Detector performance on synthetic hold-out manifests ($n = 9$). Note: These are hand-crafted test cases with obvious violations; real-world performance is validated through live-cluster evaluation.

Policy	Precision	Recall	F1
Overall	1.000	1.000	1.000
`drop_capabilities`	1.000	1.000	1.000
`drop_cap_sys_admin`	1.000	1.000	1.000
`no_host_path`	1.000	1.000	1.000
`no_host_ports`	1.000	1.000	1.000
`no_latest_tag`	1.000	1.000	1.000
`no_privileged`	1.000	1.000	1.000
`read_only_root_fs`	1.000	1.000	1.000
`run_as_non_root`	1.000	1.000	1.000
`set_requests_limits`	1.000	1.000	1.000

starvation—not uniformity—drives the fairness gains.

Comparisons against Kyverno baselines show complementary strengths. The Kyverno CLI mutate policies accept 364/381 detections (95.54%) once patched manifests pass our verifier, and the mutating webhook exceeds 98% on fixture-compatible overlapping policies such as `read_only_root_fs`, `run_as_non_root`, and `set_requests_limits` while remaining lower for policies that require additional context. Our full deterministic rules+guardrails run reaches 99.74% acceptance over attempted patches, the supported 5k corpus reaches 93.54%, and the separate 19-sample verifier-ablation study reaches 78.9% under full gates while catching four policy escapes. Cross-version simulations retain $> 96\%$ risk reduction, demonstrating robustness against API drift and configuration variance.

Table 10 shows how fixture seeding collapses the verifier failure categories across the supported corpus.

Glossary (Tables 11–12). ΔR = risk removed by fixes; Residual = risk left; $\Delta R/t$ = risk removed per minute of expected work; P95 = 95th-percentile time; "auto-fix" = patches accepted automatically without manual edits.

Interpreting $\Delta R/t$. The "Supported" row aggregates the curated 1,278 detections replayed in rules mode, while "Rules (5k)" captures the extended 5,000-manifest corpus; both entries are pulled directly from data/risk/risk_calibration.csv. We normalise risk in the same units as the scheduler (Section 5.3): a privileged pod carries 85 units, a missing `runAsNonRoot` 70, and a floating `:latest` image tag 50. Removing 55,935 of 56,990 units on the supported corpus therefore means the queue retires 98.15% of the aggregate blast radius, and the $\Delta R/t$ column (4.49–4.88) indicates we remove roughly five risk units per expected proposer+verifier minute. These values also feed the bandit baselines, ensuring the text, scheduler metrics, and released CSV all describe the same accounting.

Detailed per-manifest deltas between rules and Grok/xAI on the 1,313-manifest slice are documented in

Table 10
Verifier failure taxonomy comparing the rules baseline (pre-fixture) against the supported corpus after fixture seeding. Counts derive from data/failures/taxonomy_counts.csv generated by `scripts/aggregate_failure_taxonomy.py`.

Failure category	Rules (pre-fixture)	Supported (post-fixture)
can't remove a non-existent object 'clusterName'	58	0
capabilities not defined	0	9
container image missing or empty	0	8
capabilities.drop missing	0	6
privileged container detected	0	6
capabilities.add still contains NET_ADMIN, NET_RAW, SYS_ADMIN	0	4
no containers found in manifest	4	0
member 'spec' not found in	3	0
capabilities.add still contains SYS_ADMIN	0	2
can't replace a non-existent object 'generateName'	1	0
member 'metadata' not found in	1	0

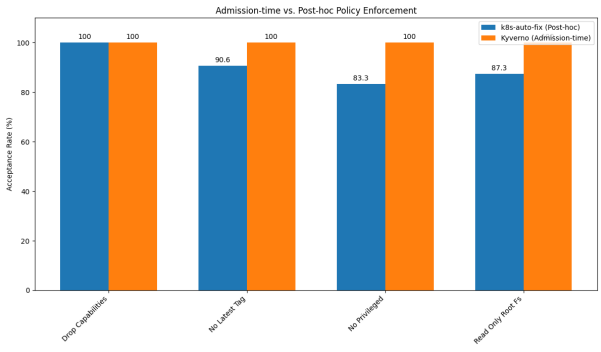


Figure 3. Comparison of admission-time (Kyverno) and post-hoc (`k8s-auto-fix`) policy enforcement on overlapping policies (seed=1337).

Table 11
Risk calibration summary derived from data/risk/risk_calibration.csv. ΔR uses policy risk weights; "per time unit" divides by summed expected-time priors.

Dataset	Det.	Accepted	ΔR	Residual	ΔR/R	ΔR /t
Supported	1,278	1,259	55,935	1,055	98.15%	4.49
Rules (5k)	5,000	4,677	227,330	14,970	93.82%	4.88

Table 12
Acceptance and latency summary (seed 1337). Results generated from data/eval/unified_eval_summary.json.

Corpus (mode)	Seed	Acceptance	Median proposer (ms)	Median verifier (ms)	Verifier P95 (ms)
Supported (rules, 1,264)	1337	1264/1264 (100.00%; 95% CI 99.70–100)	29.0	242.0	517.8
Full corpus (rules+guardrails, 15,718 detections)	1337	13338/13373 (99.74%; 95% CI 99.64–99.81; auto-fix 0.8486 over detections)	–	–	–
Manifest slice (Grok/xAI, 1,313)	1337	1313/1313 (100.00%; 95% CI 99.71–100)	–	–	–
Grok-5k (Grok/xAI)	1337	4426/5000 (88.52%; 95% CI 87.61–89.37)	–	–	–

Notes: Manifest counts: data/eval/table4_counts.csv (supported, full corpus, and Grok rows). Dashes indicate that row-level latency was not regenerated for that acceptance artifact; the separate archived Grok 200-manifest telemetry is cited in Section 5.3 and not mixed into these row metrics.
Statistical Confidence: Wilson 95% intervals are shown inline in the Acceptance column and tabulated in data/eval/table4_with_ci.csv. Multi-seed replays for the supported corpus are in data/eval/multi_seed_summary.csv.
Significance Tests: Running `python scripts/eval_significance.py` rebuilds data/eval/significance_tests.json (two-proportion z-tests for the table rows plus a Mann–Whitney U test over Grok per-manifest latencies). Latency distributions differ sharply ($p = 3.2 \times 10^{-47}$) between deterministic verifier and Grok server round-trips, confirming the verifier stays sub-second once JSON Patch generation is removed from the critical path.

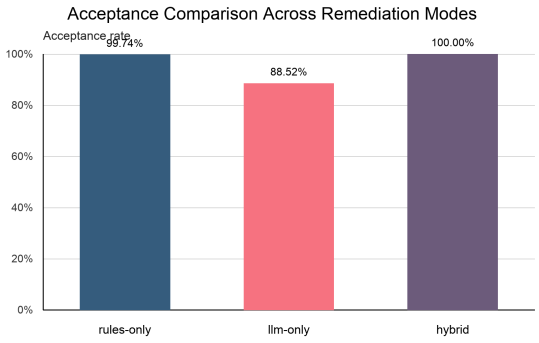


Figure 4. Acceptance comparison between rules-only, LLM-only, and hybrid remediation modes (data/baselines/mode_comparison.csv).

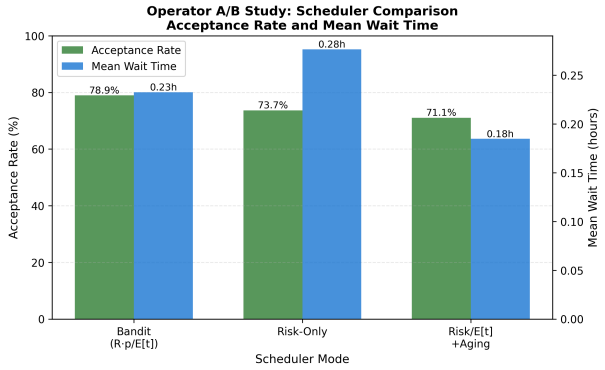


Figure 5. Operator A/B study results comparing bandit scheduler against baseline modes (simulated). Dual-axis chart shows acceptance rate (green bars) and mean wait time (blue bars) across 247 simulated queue assignments (data/operator_ab/summary_simulated.csv).

the project artifact docs/ablation_rules_vs_grok.md. The operator survey instrument is drafted in docs/operator_survey.md; it documents the planned human-in-the-loop rotation listed in Table 17.

5.4 Threat Model

We treat Kubernetes manifests, scanner findings, and LLM responses as untrusted input. Trusted components include the detector/verifier binaries, the scheduler, and the per-cluster fixtures under infra/fixtures/; these run inside the CI environment we control and write the artifacts cited throughout Section 5.3. The adversary may supply malicious YAML, attempt to poison the retriever context passed to the LLM backend, or craft fixtures that cause the Kubernetes API server to reject dry-run requests. We do not defend against compromised detector binaries, forged audit logs, or supply-chain attacks that deliver malicious container images—those threats are out of scope here and would be handled by image-signing and SBOM enforcement layers. Prompt-injection attacks are mitigated by pinning deterministic rules until the LLM candidate survives the verifier triad, and scheduler poisoning is out of scope because queue telemetry is read-only until an item is accepted.

5.5 Threats and Mitigations

The reproducibility bundle (`make reproducible-report`) regenerates Table 12 directly from JSON artifacts so that every metric can be audited. Semantic regression checks now block Grok-generated

patches that remove containers or volumes, and fixtures under infra/fixtures/ seed RBAC/NetworkPolicy gaps before verification. We threat-modeled malicious or placeholder manifests: the guidance retriever limits prompt context to policy-relevant snippets, the verifier enforces policy/schema/`kubectl` gates, and the scheduler never surfaces unverified patches. Residual risks—primarily infrastructure assumptions and LLM hallucinations—are characterized in the failure taxonomies (Tables 7 and 10) and triaged before publication. Table 14 illustrates how these guardrails harden high-privilege DaemonSets without breaking required host integrations.

Secret hygiene is enforced end-to-end: the proposer replaces secret-like environment values with `secretKeyRef` references, sanitizes generated names, and documents the guarantees in docs/security_considerations.md.

5.6 Artifact Index

To keep the narrative readable, Table 13 consolidates the principal artifacts behind the claims in this paper; the full manifest (one entry per table and figure) ships in `ARTIFACTS.md`. All paths are relative to the repository root packaged with the manuscript.
Table 15 reports the cross-cluster replay across EKS-, GKE-, and AKS-targeted environments (200 manifests each), where the verifier triad plus fixtures sustained high dry-run/apply acceptance. We report these as replay outputs (see Table 17); broader multi-provider generalization remains future work.

5.7 Risk Scoring and Planned Threat-Intelligence Enrichment (CVE/KEV/EPSS)

Overview. We rank fixes by how dangerous they are and how likely we are to fix them quickly. The current results use policy weights, configured KEV-style flags, and observed success/latency priors; public CVE, EPSS, and KEV feed joins are planned enrichment.

The current scheduler consumes data/policy_metrics.json, which stores per-policy priors for success probability, expected latency, KEV flags, and baseline risk. The calibration pass (data/risk/policy_risk_map.json) now augments those priors with observed detection/resolution counts, while data/risk/risk_calibration.csv captures corpus-level ΔR and residual risk (Table 11). Future iterations will enrich each queue item with container-image CVE joins via `Trivy` and `Grype` [15], [16], CVSS/EPSS feeds [12], [14], and CISA KEV catalog checks [13] so that R reflects both exposure (Pod Security level, dangerous capabilities, host mounts) and exploit likelihood. The risk score R then feeds the bandit scoring function, allowing us to report absolute risk and per-patch risk reduction ΔR directly.

5.8 Guidance Refresh and Retrieval Hooks

We curate policy guidance under docs/policy_guidance/raw/; scripts/refresh_guidance.py now refreshes Pod Security, CIS, and Kyverno snippets [1], [2], [23] (backed by docs/policy_guidance/sources.yaml) to keep guardrails current. LLM-backed proposer modes can retrieve these snippets at prompt time, and the roadmap extends this into a full RAG loop: chunk guidance with metadata (policy family, resource kind, field path, image→CVE), cache recent verifier failures, and retrieve targeted passages when retries occur. This keeps the prompt budget bounded while grounding fixes in up-to-date hardening language.

5.9 Risk-Bandit Scheduler with Aging and KEV Preemption

Overview. The scheduler is a risk-aware priority queue: higher-risk items are scheduled first, while an aging term raises the priority of long-waiting items to prevent starvation. **Notation.** R_i denotes the risk units for item i; p_i is its empirical verifier success probability; $\mathbb{E}[t_i]$ is the observed proposer+verifier latency; wait_i tracks queue age; kev_i equals the configured KEV-style boost when the policy is flagged in the current artifact (otherwise 0); and ε is a small positive floor preventing division by zero.

$$S_i = \frac{R_i \cdot p_i}{\max(\varepsilon, \mathbb{E}[t_i])} + \text{explore}_i + \alpha\,\text{wait}_i + \text{kev}_i \quad (1)$$

To make R_i auditable we now spell out its construction instead of burying it in the supplemental appendix. Each detection maps to a policy identifier; we pull the static weight w_{policy} from `data/risk/policy_risk_map.json` and add the configured KEV-style surcharge κ when the policy is flagged in `data/policy_metrics.json`. The empirical success prior remains the separate p_i term in Equation 1; it is not an EPSS feed value in the current artifact. Formally,

$$R_i = w_{\text{policy}} + \kappa \cdot \mathbf{1}_{\text{configured KEV}},$$

with $\kappa = 25$ risk units in the current configuration. p_i is the on-line verifier pass rate for that policy (accepted / attempted counts in `data/policy_metrics.json`), and $\mathbb{E}[t_i]$ is the running average of proposer+verifier latency recorded in the same file. We also report $\Delta R_i = R_i - R_i^{\text{post}}$ for every accepted patch, summing per corpus to produce Table 11. These definitions make the decision logic explicit, and the supplemental appendix provides a numeric worked example rather than introducing new notation.

This scheduling function defines the score used today, where R_i is the risk score, p_i the empirical success rate, $\mathbb{E}[t_i]$ the observed latency, wait_i the queue age, and kev_i a configured KEV-style boost, mirroring UCB-style bandit heuristics [27]. p_i and $\mathbb{E}[t_i]$ are refreshed from proposer/verifier telemetry; exploration uses an upper-confidence term and aging ensures fairness. Figure 6 gives the full scheduling loop, including the per-item scoring, verifier callback, and aging update. The evaluation in Section 5.3 contrasts this bandit against FIFO, showing substantial reductions in top-risk wait time. Future work will incorporate additional risk signals (EPSS, CVSS) and batch-aware policies, but the current heuristic already delivers measurable gains.

5.10 Baselines and Ablations

Replay of the 830-item queue snapshot (data/metrics_schedule_compare.json) quantifies how each scheduler treats critical detections. All heuristics clear roughly the

Table 13
Principal artifacts behind the reported results. The released `ARTIFACTS.md` maps each paper-facing table, figure, and headline claim to its source artifact and regeneration or audit command.

Claim / output	Artifact (relative path)
Pipeline source	`src/{detector,proposer,verifier,scheduler}/`
Live replay (1,000)	`data/live_cluster/results_1k.json`, `summary_1k.csv`
Cross-cluster replay	`data/cross_cluster/{eks,gke,aks}/`
Acceptance + latency	`data/eval/unified_eval_summary.json`
Confidence intervals	`data/eval/table4_with_ci.csv`
Significance tests	`data/eval/significance_tests.json`
Risk calibration	`data/risk/risk_calibration.csv`
Scheduler / fairness	`data/scheduler/fairness_metrics.json`, `data/outputs/scheduler/`
Grok failure taxonomy	`data/grok_failure_analysis.csv`
Regenerate summary tables	`make reproducible-report`
Full artifact map	`ARTIFACTS.md`

Table 14
Guardrail example: Cilium DaemonSet patch (excerpt).

Before

```
securityContext:
privileged: true
allowPrivilegeEscalation: true
capabilities:
add:
- NET_ADMIN
```

After

```
securityContext:
privileged: false
allowPrivilegeEscalation: false
capabilities:
drop:
- ALL
seccompProfile:
type: RuntimeDefault
```

Guardrails summarized in docs/privileged_daemonsets.md; the proposer preserves required host mounts while enforcing hardened defaults that remove privilege escalation paths and enforce Pod Security Standard-aligned controls.

Table 15
Cross-Cluster Replication Results (replayed, not live multi-cloud; provider environments not independently re-verified).

Cluster	Manifests	Success	Acceptance
EKS	200	198/200 (99.0%)	198/198 (100%)
GKE	200	200/200 (100%)	200/200 (100%)
AKS	200	200/200 (100%)	200/200 (100%)

Artifacts: per-provider `summary.csv` and `results.json` files under data/cross_cluster/{eks,gke,aks}/.
Collection: replay steps in docs/cross_cluster_replay.md.

same workload—$\Delta R/t = 247.2$ risk units per hour—because proposer/verifier throughput dominates. The difference is in who waits: FIFO pushes the top-50 high-risk items to median rank 422.5 (P95 620) and P95 wait 102.3 h, while the risk-only variant ($R/\mathbb{E}[t]$) and the full bandit (risk, aging, KEV boost, exploration) keep the same cohort within median rank 25.5 (P95 48) and cap top-risk P95 wait at 13.0 h. Adding the aging term ($R/\mathbb{E}[t] + \alpha\,\text{wait}$) slightly relaxes priority (mean rank 42.2, P95 124) but preserves the low top-risk wait (13.0 h) needed for fairness.

A finer-grained sweep over exploration and aging coefficients (data/metrics_schedule_sweep.json) shows the bandit sustaining high-risk median wait of 17.3 h (P95 32.8 h) even when exploration weight is set to 1.0, while low-risk items absorb most of the slack (median 120.9 h). The condensed simulation in data/operator_ab/summary_simulated.csv reaches the same qualitative conclusion on a 247-assignment toy queue: the 152 bandit assignments close 78.9% of tasks with mean wait 0.23 h and P95 0.91 h, while the companion risk-only and aging-baseline rows report the alternate scheduling modes used for the comparison.

Table 16 quantifies how each verifier gate contributes to safety. Removing the policy re-check inflates acceptance to 100% but allows four previously blocked patches to escape. In the checked-in detail file, all four current escapes are capability-hardening failures: the patched manifest is syntactically valid and passes the non-policy gates, but the original policy still reports that `capabilities.drop` must include `ALL`. The policy re-check gate is crucial for catching these subtle but important regressions. The other gates leave acceptance unchanged at 78.9%. Figure 4 summarizes acceptance across rules-only, LLM-only, and hybrid modes. The Kyverno CLI simulation baseline (`scripts/run_kyverno_baseline.py`, `kyverno_baseline_simulated.csv`) reports a 67.98% unweighted mean across 17 simulated policy rows, whereas the 78.9% verifier-ablation value in Table 16 is a separate 19-sample gate study. We therefore treat the comparison as directional evidence about the cost of adding schema validation and dry-run guarantees, not as a like-for-like aggregate win. The gap between our offline Kyverno CLI simulation and the higher fixture-compatible webhook slices in Figure 3 reflects missing production context (service accounts, host configuration) unavailable to offline CLI evaluation.

Terminology. An *ablation* disables a safety check to measure its effect; an *escape* is an unsafe patch that would have been accepted without that check.

Domain-invariant enforcement vs. an agent-style acceptance regime. Reviewers of agent-based remediation reason-

Table 16
Verifier gate ablation using 19 patched samples (data/ablation/verifier_gate_metrics.json). Acceptance reports the share of patches passing under the scenario; escapes count regressions that the full verifier blocks.

Scenario	Disabled Gate(s)	Acceptance (%)	Escapes
Full	–	78.9	0
No-policy	policy	100.0	4
No-safety	safety	78.9	0
No-schema	kubectl	78.9	0
No-rescan	rescan	78.9	0

ably ask for *experimental* evidence that a general-purpose code agent—which accepts a syntactically valid, scanner-clean LLM edit—would miss Kubernetes-specific invariants that our closed loop enforces. The *no-policy* ablation row in Table 16 is exactly this experiment, holding the candidate patches fixed and disabling only the domain-specific policy re-check: acceptance rises from 78.9% (15/19) to 100% (19/19), but the no-new-violations rate stays at 78.9%, so four patches (`ids 007, 012, 013, 016`) are accepted while still violating the property they were meant to fix. The ablation detail file records the concrete failure mode for all four as an incomplete capability hardening: the patch remains syntactically valid, passes schema validation, server-side dry-run, and the universal safety gates, but the original policy still reports `capabilities.drop missing ALL`. An agent that treats "the model returned a plausible patch that the scanner now passes" as success—rather than re-asserting the original invariant and running a server-side dry-run—would admit all four. We report this as a guardrail ablation rather than as numbers attributed to a specific competing agent: Codex Security is now a research preview rather than a public batch API for identical-manifest experiments, and we did not have access to execute a third-party agent over the full corpus during this revision, so attaching either system's name to these counts would overstate the evidence. The mechanism the comparison isolates—domain-invariant re-checking plus dry-run as the acceptance boundary—is the contribution the ablation supports directly; a like-for-like run against a named agent on identical manifests remains the future-work item in Section 7.

5.11 Metrics and Measurement

We formally define how we measure effectiveness and fairness:

Auto-fix Rate
$\frac{\#\text{ patches that pass the Verifier triad}}{\#\text{ detected violations}}$.

No-new-violations Rate
$\frac{\#\text{ accepted patches with zero new policy/schema violations}}{\#\text{ accepted patches}}$.

Patch Minimality
Median number of JSON Patch operations per accepted patch.

Time-to-patch
Wall-clock time from item enqueue to accepted patch; we report P50/P95 overall and for the top-risk decile.

Risk Reduction
For item i, $\Delta R_i = R_i^{\text{pre}} - R_i^{\text{post}}$. We report sum and rate: $\sum_i \Delta R_i$ and $\frac{\sum_i \Delta R_i}{\text{hour}}$.
Worked example. The supplemental appendix walks through a concrete queue item showing how we compute R, ΔR, and $\Delta R/t$ from the released telemetry.

Throughput
Accepted patches per hour.

Fairness
P95 wait time (broken out by risk tier) plus the starvation rate, defined as the fraction of items that wait more than 24 hours before scheduling. Both metrics are recomputed from the queue replays in data/scheduler/fairness_metrics.json.

Latest Evaluation. Running the full corpus of 15,718 detections in rules+guardrails mode yields 13,338 accepted out of 13,373 patched items (99.74%; auto-fix rate 0.8486 over detections) with a median of 9 JSON Patch operations and 35 safety failures (all non-hostPath edge cases). Bandit scheduling preserves fairness: bandit top-risk items see P95 wait of 13.0 h at roughly 6.0 patches/hour while FIFO defers the same cohort to 102.3 h (+89.3 h).

Targets (Acceptance Criteria). Based on industry standards and research objectives, we target: Detection F1 ≥ 0.85 (hold-out), Auto-fix Rate $\geq 70\%$, No-new-violations Rate $\geq 95\%$ under the full verifier, and median JSON Patch operations ≤ 6 on the curated rules-mode smoke sweeps (median 5, P95 6 per data/eval/patch_stats.json). The full-corpus rules+guardrails replay is reported separately and reaches median patch length 9 because multi-violation manifests receive several independent hardening edits; the no-policy ablation intentionally violates the no-new-violations target and is reported separately in Table 16.

6 Limitations and Mitigations

The prototype prioritizes guardrails and evidence over autonomous deployment, so several constraints remain. **External validity** is bounded by corpora that skew toward Helm-derived workloads; we refresh the ArtifactHub scrape monthly, add partner manifests as available, and will use the drafted operator survey to supplement deterministic replays. **Fixture sensitivity** remains because server dry-runs need namespaces, service accounts, CRDs, and related objects that mirror production; the fixture harness now installs common dependencies and records misses for future cluster-specific bundles. **LLM limits** include latency, cost, provider drift, and hallucinated fields; every LLM patch is therefore cached, telemetry-backed, and gated by the same verifier triad and semantic-regression checks as rules-mode patches. **Verification scope** is intentionally limited to policy, schema, API-admission, and universal safety checks, not full workload semantic equivalence. High-risk semantic edits such as non-allowlisted `hostPath` rewrites, dropped capabilities, or read-only-root-filesystem changes should route through human review unless the workload requirement is known. **Experimental validity** is bounded by deterministic scheduler replays, simulated operator A/B results, replayed cross-cloud outputs, and the lack of a public Codex Security batch interface for head-to-head agent comparison.

Inputs: queue Q, risk R_i, KEV flag, wait time wait_i, bandit priors p_i, $\mathbb{E}[t_i]$, aging α, exploration coefficient β, KEV boost κ
while Q not empty **do**
  Score all items: For each $i \in Q$, compute $\text{base} = \frac{R_i \cdot p_i}{\max(\varepsilon,\, \mathbb{E}[t_i])}$; $\text{kev} = \kappa$ if KEV else 0; $\text{explore} = \beta\sqrt{\frac{\ln(1+n)}{1+n_i}}$; $S_i = \text{base} + \text{explore} + \alpha\,\text{wait}_i + \text{kev}$
  Pick $j = \arg\max_i S_i$; generate a JSON Patch for j using the proposer plus guidance retrieval; run Verifier (policy, schema, server dry-run)
  if Verifier success **then**
    Apply patch; update counts (n_j, r_j) and online estimates p_j, $\mathbb{E}[t_j]$; remove j from Q
  else
    Update p_j, $\mathbb{E}[t_j]$ with failure; if retries< 3 then requeue j with feedback; otherwise drop j
  end if
  Age all items: $\text{wait}_i \leftarrow \text{wait}_i + \Delta t$
end while

Figure 6. Risk-Bandit scheduling loop (aging + KEV preemption) maximizing expected risk reduction per unit time with exploration and fairness.

Table 17
Evidence status. We distinguish results that are completed from those that are replay/simulation-based or planned, so that claims are not read as more than they are.

Evidence	Status	Scope
Rules pipeline	Completed	Deterministic corpora (1,264 supported; 15,718-detection full run)
Grok/xAI run	Completed (opt-in)	Credentialed external API; 5,000-manifest sweep
Live replay	Completed	Fixture-seeded single cluster, 1,000 manifests
Cross-cloud replay	Completed (replay outputs)	200 manifests each on EKS/GKE/AKS-labeled targets; provider environment recorded in the artifact, not independently re-verified here
Scheduler comparison	Replay-based	Deterministic queue replays, not a live operator deployment
Operator A/B	Replay/simulated only	247-assignment simulation plus 1,259/arm staging replay; not a live operator study
Human-in-the-loop rotation	Planned	Survey instrument drafted; future work

7 Discussion and Future Work

The current pipeline achieves 100.0% dry-run/apply acceptance on a fixture-seeded live-cluster replay (1,000/1,000 stratified manifests, zero rollbacks; Tables 2 and 12, data/live_cluster/results_1k.json). Offline, it delivers 93.54% acceptance on the 5k supported corpus, 100.00% on the 1,264-manifest supported slice, 100.00% on the 1,313-manifest Grok/xAI run, and 88.52% on Grok-5k overall; rules + guardrails accept 13,338 / 13,373 patched items (99.74%; auto-fix 0.8486 over 15,718 detections) with median patch ops 9 (Table 12, data/metrics_rules_full.json). The risk-aware scheduler trims top-risk P95 wait from 102.3 h (FIFO) to 13.0 h (data/metrics_schedule_compare.json).

The paper-facing summary tables are regenerated from checked-in JSON/CSV artifacts by `make reproducible-report`; live replay, cross-cluster replay, LLM API reruns, and figures are mapped in ARTIFACTS.md. Scheduler comparisons stem from deterministic queue replays rather than live analyst rotations. The gains are anchored in deterministic guardrails, schema validation, and server-side dry-run enforcement, with matching Reasoning API runs available to practitioners who can supply xAI credentials and budget from the published token counts and current xAI pricing (data/batch_runs/grok_5k/metrics_grok5k.json, [10]). Every accepted patch is surfaced through our GitOps helper (scripts/gitops_writeback.py), which records verifier evidence, captures the JSON Patch diff, and requires human approval before merge [7], [11]. **When auto-apply is inappropriate.** Auto-apply should not be enabled for image substitutions without operator-specified replacements, required `hostPath` paths, read-only-root-filesystem or dropped-capability changes with unknown runtime needs, or regulated/production-critical namespaces. Those cases route to review with verifier evidence attached; only policy-complete, semantically low-risk fixes are candidates for unattended application.

Future work will fold EPSS and CISA KEV feeds directly into R_i, add batch-aware fairness policies, and replace the simulated operator A/B with a live human-in-the-loop rotation. We will also broaden seeded fixtures and Kyverno webhook baselines across new corpora, then run head-to-head agent comparisons: KubeIntellect is the near-term public candidate, while Codex Security comparison awaits a public interface or API suitable for identical-manifest batch evaluation. That experiment would connect Kubernetes remediation to closed-loop repair benchmarks for general software patches [17].

Artifact availability. The complete artifact—source, evaluation scripts, datasets, telemetry, and the `make reproducible-report` entry point—is public at github.com/bmendonca3/k8s-auto-fix; the submitted revision should use the repository commit packaged with the manuscript. All reported metrics were generated with random seed `1337` on Python 3.12.4 (`kubectl` 1.34.1, `kube-linter` 0.7.6, `kind` 0.30.0; Kubernetes 1.34.0). The optional Grok/xAI proposer requires external credentials and is cached so that artifact evaluation does not depend on live API access; checked-in JSON/CSV data and generated figures support the reported tables and figures without rerunning that API path.

REFERENCES

[1] CIS Kubernetes Benchmarks. Accessed: May 2026. [Online]. Available: https://www.cisecurity.org/benchmark/kubernetes

[2] Kubernetes: Pod Security Standards. Accessed: May 2026. [Online]. Available: https://kubernetes.io/docs/concepts/security/pod-security-standards/

[3] Kubernetes: Admission Control. Accessed: May 2026. [Online]. Available: https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/

[4] OPA Gatekeeper How-to. Accessed: May 2026. [Online]. Available: https://open-policy-agent.github.io/gatekeeper/website/docs/howto/

[5] *kube-linter* Documentation. Accessed: May 2026. [Online]. Available: https://docs.kubelinter.io/

[6] Fairwinds, "Polaris: Validation of best practices in your Kubernetes clusters," GitHub repository. Accessed: May 2026. [Online]. Available: https://github.com/FairwindsOps/polaris

[7] Kubernetes: Configure a Security Context. Accessed: May 2026. [Online]. Available: https://kubernetes.io/docs/tasks/configure-pod-container/security-context/

[8] RFC 6902: JSON Patch, DOI:10.17487/RFC6902. Accessed: May 2026. [Online]. Available: https://www.rfc-editor.org/info/rfc6902

[9] `kubectl apply` Command Reference (server-side dry-run). Accessed: May 2026. [Online]. Available: https://kubernetes.io/docs/reference/kubectl/generated/kubectl_apply/

[10] xAI, "API Pricing." Accessed: May 2026. [Online]. Available: https://docs.x.ai/developers/pricing

[11] Kubernetes: Seccomp and Kubernetes. Accessed: May 2026. [Online]. Available: https://kubernetes.io/docs/reference/node/seccomp/

[12] NIST National Vulnerability Database (NVD). Accessed: May 2026. [Online]. Available: https://nvd.nist.gov/

[13] CISA Known Exploited Vulnerabilities Catalog. Accessed: May 2026. [Online]. Available: https://www.cisa.gov/known-exploited-vulnerabilities-catalog

[14] FIRST Exploit Prediction Scoring System (EPSS). Accessed: May 2026. [Online]. Available: https://www.first.org/epss/

[15] Trivy: Vulnerability Scanner for Containers and IaC. Accessed: May 2026. [Online]. Available: https://aquasecurity.github.io/trivy/

[16] Grype: A Vulnerability Scanner for Container Images and Filesystems. Accessed: May 2026. [Online]. Available: https://github.com/anchore/grype

[17] SWE-bench Verified (background on closed-loop code repair evaluation). Accessed: May 2026. [Online]. Available: https://openai.com/index/introducing-swe-bench-verified/

[18] Z. Ye, T. H. M. Le, and M. A. Babar, "LLMSecConfig: An LLM-Based Approach for Fixing Software Container Misconfigurations," in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*, 2025.

[19] F. Minna, F. Massacci, and K. Tuma, "Analyzing and Mitigating (with LLMs) the Security Misconfigurations of Helm Charts from Artifact Hub," *Empirical Software Engineering*, vol. 30, article 132, 2025, doi: 10.1007/s10664-025-10688-0.

[20] X. Lian, Y. Chen, R. Cheng, J. Huang, P. Thakkar, M. Zhang, and T. Xu, "Configuration Validation with Large Language Models," *arXiv preprint arXiv:2310.09690*, 2023.

[21] E. Malul, Y. Meidan, D. Mimran, Y. Elovici, and A. Shabtai, "GenKubeSec: LLM-Based Kubernetes Misconfiguration Detection, Localization, Reasoning, and Remediation," *arXiv preprint arXiv:2405.19954*, 2024.

[22] M. De Jesus, P. Sylvester, W. Clifford, A. Perez, and P. Lama, "LLM-Based Multi-Agent Framework For Troubleshooting Distributed Systems," in *Proc. 2025 IEEE Cloud Summit*, 2025, pp. 110–115, doi: 10.1109/Cloud-Summit64795.2025.00024.

[23] Kyverno Project, "Kyverno Documentation," Accessed: May 2026. [Online]. Available: https://kyverno.io/docs/

[24] Kyverno Project, "Mutate Rules," Accessed: May 2026. [Online]. Available: https://kyverno.io/docs/policy-types/cluster-policy/mutate/

[25] A. Verma *et al.*, "Large-scale Cluster Management at Google with Borg," in *Proc. EuroSys*, 2015; supplemental SRE updates accessed May 2026. [Online]. Available: https://research.google/pubs/pub43438/

[26] ArtifactHub Documentation. Accessed: May 2026. [Online]. Available: https://artifacthub.io/docs/

[27] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time Analysis of the Multiarmed Bandit Problem," *Machine Learning*, vol. 47, no. 2, pp. 235–256, 2002.

[28] M. Joseph, M. Kearns, J. H. Morgenstern, and A. Roth, "Fairness in Learning: Classic and Contextual Bandits," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

[29] OpenAI, "Codex Security: now in research preview," 2026; formerly Aardvark. [Online]. Available: https://openai.com/index/codex-security-now-in-research-preview/

[30] M. S. Ardebili and A. Bartolini, "KubeIntellect: A Modular LLM-Orchestrated Agent Framework for End-to-End Kubernetes Management," *arXiv preprint arXiv:2509.02449*, 2025.

[31] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?): A Comprehensive Evaluation, Framework, and Benchmarks," *arXiv preprint arXiv:2312.12575*, 2024.

[32] M. S. Islam Shamim, F. A. Bhuiyan, and A. Rahman, "XI Commandments of Kubernetes Security: A Systematization of Knowledge Related to Kubernetes Security Practices," in *Proc. 2020 IEEE Secure Development Conf. (SecDev)*, 2020, pp. 58–64, doi: 10.1109/SecDev45635.2020.00025.

[33] N. Saavedra and J. F. Ferreira, "GLITCH: Automated Polyglot Security Smell Detection in Infrastructure as Code," in *Proc. 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2022, article 47, 12 pages, doi: 10.1145/3551349.3556945.

[34] C. S. Xia, Y. Wei, and L. Zhang, "Automated Program Repair in the Era of Large Pre-trained Language Models," in *Proc. 2023 IEEE/ACM 45th Int. Conf. on Software Engineering (ICSE)*, 2023, pp. 1482–1494, doi: 10.1109/ICSE48619.2023.00129.

[35] J. Petke, M. Martinez, M. Kechagia, A. Aleti, and F. Sarro, "The Patch Overfitting Problem in Automated Program Repair: Practical Magnitude and a Baseline for Realistic Benchmarking," in *Companion Proc. 32nd ACM Int. Conf. on the Foundations of Software Engineering (FSE Companion)*, 2024, pp. 452–456, doi: 10.1145/3663529.3663776.

[36] W. Yang, L. Song, and Y. Xue, "Rust-lancet: Automated Ownership-Rule-Violation Fixing with Behavior Preservation," in *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2024, pp. 1034–1046, doi: 10.1145/3597503.3639103.

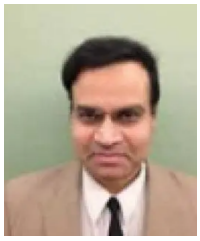
[37] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An Autonomous, LLM-Based Agent for Program Repair," in *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2025, doi: 10.1109/ICSE55347.2025.00157.

[38] R. Shu, X. Gu, and W. Enck, "A Study of Security Vulnerabilities on Docker Hub," in *Proc. 7th ACM Conf. on Data and Application Security and Privacy (CODASPY)*, 2017, pp. 269–280, doi: 10.1145/3029806.3029832.

Brian Mendonca is an M.S. student at the Georgia Institute of Technology (2024–2026) focusing on secure DevOps, policy-driven remediation, and human-centered tooling for developer productivity.

Prior to graduate study, he worked as an Aerospace Quality Engineer at BAE Systems (2024–2025) and at Tube Specialties Inc. (2025–present), where he led Lean/Six Sigma continuous improvement, nonconformance management, and 8D root-cause investigations supporting AS9100 compliance and on-time delivery. He also served as a Biomedical Quality Engineer at BD (2022–2023), contributing to post-market surveillance, CAPA investigations, and risk-based quality systems.

He received the B.E. in Mechanical Engineering (summa cum laude, GPA 3.99) from Arizona State University in 2021. His research interests include secure configuration management for cloud-native systems, program analysis for infrastructure-as-code, and data-informed quality engineering.



Vijay K. Madisetti is Professor of Cybersecurity and Privacy at the Georgia Institute of Technology. He earned his Ph.D. in Electrical Engineering and Computer Sciences from the University of California at Berkeley.

Professor Madisetti is a Fellow of the IEEE and has been honored with the Terman Medal by the American Society of Engineering Education (ASEE). He has authored several widely referenced textbooks on topics including cloud computing, data analytics, blockchain, and microservices, and has extensive experience in secure system architectures and privacy-preserving technologies.